

Templates

Unit-07

```

#include <iostream>
using namespace std;

// Template function declaration
template <typename T>
T myMax(T x, T y) {
    return (x > y)? x : y;
}

int main() {
    // Using the template function with int data type
    cout << "Max(10, 15): " << myMax<int>(10, 15) << endl;

    // Using the template function with double data type
    cout << "Max(5.5, 2.5): " << myMax<double>(5.5, 2.5) << endl;

    // The compiler can also deduce the data type automatically
    cout << "Max('k', 's'): " << myMax('k', 's') << endl;

    return 0;
}

```

In this example:

template <typename T> is the template parameter declaration.

T myMax(T x, T y) defines a function myMax that takes two parameters of type T and returns a value of type T.

Note that in some cases, you do not need to explicitly specify the type (like for myMax('k', 's')) because the compiler can deduce the type based on the arguments passed to the function.

Template functions are powerful and flexible, making them a fundamental feature of C++ that supports generic programming.

```
#include <iostream>
#include <string>
using namespace std;

// Template function with two different typenames
template <typename T1, typename T2>
string compare(T1 a, T2 b) {
    if (a == b)
        return "Equal";
    else
        return "Not Equal";
}
```

What is happening here?

```
int main() {
    cout << "Compare 5 and 5.0: " << compare(5, 5.0) << endl; // Implicit conversion
    cout << "Compare 'a' and 97: " << compare('a', 97) << endl; // 'a' is ASCII 97
    cout << "Compare 10.5 and '10': " << compare(10.5, '10') << endl; // Comparing di
    return 0;
}
```

```
template <typename T>
struct Box {
    T contents;
    void setContents(T newContents) {
        contents = newContents;
    }
    T getContents() {
        return contents;
    }
};

int main() {
    Box<int> intBox;
    intBox.setContents(123);
    cout << "Integer Box contains: " << intBox.getContents() << endl;

    Box<string> stringBox;
    stringBox.setContents("Hello Templates");
    cout << "String Box contains: " << stringBox.getContents() << endl;
}
```

```
template <typename T>
struct Array {
    T* arr;
    Array() { arr = new T[10]; } // Simple constructor
    // Other members
};
```

// Specialization for int type

```
template <>
struct Array<int> {
    int* arr;
    Array() { arr = new int[20]; } // Different constructor for int
    // Other members
};
```

Template Specialization

Templates with Multiple Types Using Struct

cpp

```
template <typename K, typename V>
struct MapEntry {
    K key;
    V value;
    // Constructor, getters, setters
};

int main() {
    MapEntry<string, int> ageEntry;
    ageEntry.key = "Alice";
    ageEntry.value = 30;
    // Use the MapEntry
}
```

Non-Type Template Parameters in Structs

cpp

```
template <typename T, int size>
struct FixedArray {
    T array[size];
    // Other members
};

int main() {
    FixedArray<int, 5> myArray;
    // Use the FixedArray
    for(int i=0; i<5; i++){
        myArray[i] = i * i;
    }
    for(int i=0; i<5; i++) cout << myArray[i] << ", ";
    cout << endl;
```

Can you find a bug...?

Non-type template parameters in C++ templates allow you to pass values, not just types, to your templates. This feature enables templates to be more flexible and powerful, as you can define behavior not just based on types but also on certain compile-time constants or other value-based parameters. These parameters can be integral types (like `int`, `unsigned`, `size_t`), pointers, references, or even user-defined constexpr objects that have a constant value at compile time.

Non-Type Template Parameters in Structs

cpp

```
template <typename T, int size>
struct FixedArray {
    T array[size];
    // Other members
};

int main() {
    FixedArray<int, 5> myArray;
    // Use the FixedArray
    for(int i=0; i<5; i++){
        myArray.array[i] = i * i;
    }
    for(int i=0; i<5; i++) cout << myArray.array[i] << ", ";
    cout << endl;
```

Non-type template parameters in C++ templates allow you to pass values, not just types, to your templates. This feature enables templates to be more flexible and powerful, as you can define behavior not just based on types but also on certain compile-time constants or other value-based parameters. These parameters can be integral types (like `int`, `unsigned`, `size_t`), pointers, references, or even user-defined constexpr objects that have a constant value at compile time.

Non-Type Template Parameters in Structs

cpp

```
template <typename T, int size>
struct FixedArray {
    T array[size];
    T& operator[](int index) {
        return array[index];
    }
    // Other members
};

int main() {
    FixedArray<int, 5> myArray;
    // Use the FixedArray
}
```

Non-type template parameters in C++ templates allow you to pass values, not just types, to your templates. This feature enables templates to be more flexible and powerful, as you can define behavior not just based on types but also on certain compile-time constants or other value-based parameters. These parameters can be integral types (like `int`, `unsigned`, `size_t`), pointers, references, or even user-defined constexpr objects that have a constant value at compile time.

Non-Type Template Parameters in Structs

cpp

```
template <typename T, int size>
struct FixedArray {
    T array[size];
    T& operator[](int index) {
        return array[index];
    }
    // Other members
};

int main() {
    FixedArray<int, 5> myArray;
    for(int i=0; i<5; i++){
        myArray[i] = i * i;
    }
    for(int i=0; i<5; i++) cout << myArray[i] << ", ";
    cout << endl;
```

Non-type template parameters in C++ templates allow you to pass values, not just types, to your templates. This feature enables templates to be more flexible and powerful, as you can define behavior not just based on types but also on certain compile-time constants or other value-based parameters. These parameters can be integral types (like `int`, `unsigned`, `size_t`), pointers, references, or even user-defined constexpr objects that have a constant value at compile time.

```
#include <iostream>
```

```
// Base case for the recursive print function template.
```

```
// Needed to end the recursion.
```

```
void print() {
```

```
    std::cout << std::endl;
```

```
}
```

```
// Variadic template function that accepts any number of arguments.
```

```
template <typename T, typename... Args>
```

```
void print(T first, Args... args) {
```

```
    std::cout << first << " ";
```

```
    print(args...); // Recursive call with one fewer argument.
```

```
}
```

```
int main() {
```

```
    print(1, 2, 3.14, "Hello", 'c');
```

```
    return 0;
```

```
}
```

Variadic Template